SIGNAL LOST

A Fog-of-War Stealth Strategy Game

A Course Project Report

Submitted in the partial fulfillment of the completion of the course

SOFTWARE ENGINEERING AND SYSTEM DESIGN

(24CS301PC311)

In III Year, I Semester B. Tech-CSE

Submitted by

Uzma Yasmeen

(Hall Ticket Number)

Under the supervision of the faculty

Dr. <Faculty Name>

<Assistant or Associate Professor>,

School of Computer Science & Artificial Intelligence

2026-2027

Department of Computer Science & Artificial Intelligence

SR University

Warangal, Telangana, India

INDEX

1. Abstract
2. Introduction
    2.1 Problem Statement
    2.2 Objectives
    2.3 Scope
3. Software Requirements Specification
    3.1 Functional Requirements
    3.2 Non-Functional Requirements
    3.3 Hardware and Software Requirements
4. System Analysis
    4.1 Existing System
    4.2 Proposed System
5. System Design
    5.1 System Architecture
    5.2 Use Case Diagram
    5.3 Class Diagram
    5.4 Sequence Diagram
    5.5 Activity Diagram
    5.6 Guard State Machine
    5.7 Deployment View
6. Methodology
    6.1 Development approach
    6.2 Version control
    6.3 Testing strategy
    6.4 Verification by exhaustive search
7. Implementation
    7.1 Module organisation
    7.2 Fog of war
    7.3 Pathfinding and pursuit
    7.4 Undo
    7.5 Saved progress and artwork
    7.6 The browser build and the installable web app
    7.7 The Android build
    7.8 The leaderboard service
    7.9 What playtesting found
8. Output Screenshots
9. Conclusion
    9.1 Summary
    9.2 What the project demonstrates
    9.3 Limitations
    9.4 Future work
10. References

1. Abstract

Signal Lost is a turn-based stealth strategy game developed in Java. The player must escape a facility that is hidden by a fog of war, collecting keys and reaching an exit while avoiding guards. Visibility is computed as a breadth-first flood fill from the player's position, so that walls block sight and tiles already visited remain on the map as a dimmer memory. Guards patrol fixed routes and raise an alarm when they see the player, and a hunter responds to that alarm by pursuing the player using A* search.

The project was undertaken to demonstrate graph algorithms, design patterns and layered architecture in a single application. Because the game is turn-based and deterministic, every decision it makes is plain logic that can be tested. Five design patterns are used: Command for undoable turns, Strategy for guard behaviours, State for guard modes, Observer for the alarm, and Factory for guards and levels.

The completed system has ten levels, six playable characters, three lives per run, a limited hint system, a star rating based on turns taken and times seen, and saved progress for each player. It runs on four front ends — desktop, browser, installable web app and Android — over one shared rules core. It is verified by 260 automated tests in 31 test classes, with a further 18 covering the leaderboard service described in Section 7.7. These include an architectural test that fails the build if the game rules ever refer to the user interface, a reachability test that proves every key and exit can be walked to on all ten levels, and a fairness test that proves cover is always within reach of a hunter. Full winnability against moving guards was established by exhaustive search of the game tree for level 1; Section 6.4 explains why that search does not scale to the later levels and what was done instead.

Keywords: fog of war, breadth-first search, A* pathfinding, design patterns, layered architecture, automated testing.

2. Introduction

2.1 Problem Statement

Most student game projects are built in real time, and real time hides the software engineering. When a character moves across the screen sixty times a second, the important decisions — what can be seen, where an opponent moves, whether a rule was broken — are buried inside animation code. Such a project is difficult to explain, difficult to draw in UML, and very difficult to unit test, because almost nothing can be separated from the rendering that drives it.

There is a second problem with games built this way. Nobody can prove they are playable. A level is considered finished when its author manages to complete it, which treats "difficult" and "impossible" as the same thing. A level whose only key sits behind a door that never opens will pass every unit test written for it, because unit tests check that the rules behave correctly, not that the game built from those rules can be won.

Signal Lost addresses both problems. The game is turn-based, so every decision is a separate step rather than a frame of animation. The rules are written without any reference to the display, so they can be tested without opening a window. And because the game is deterministic, it can be searched rather than only played, which allows the question "can this level be completed?" to be answered directly.

2.2 Objectives

1. Implement fog of war as a graph traversal, using a breadth-first flood fill bounded by a sight radius and blocked by walls, with seen terrain kept as memory separate from current visibility.
2. Implement guard pursuit using A* search with a Manhattan distance heuristic and a priority queue, with deterministic tie-breaking.
3. Apply design patterns where they are genuinely needed rather than for display: Command, Strategy, State, Observer and Factory.
4. Keep the game rules independent of the user interface, and enforce that independence automatically.
5. Verify behaviour by automated tests covering visibility, pathfinding, undo, detection, capture, scoring and saved progress.
6. Verify playability as far as it can be verified: reachability on every level, and exhaustive search of the game tree where the state space allows.
7. Deliver a complete, playable game rather than a technical demonstration.

2.3 Scope

The project covers a single-player desktop game with a grid-based level model, breadth-first visibility with fog memory, patrolling and hunting guards with a three-state mode machine, an observer-driven alarm, A* pathfinding, a command stack supporting exact undo, collectible keys that unlock an exit, three lives per run, a limited hint system, a star rating, ten levels with star-based unlocking, saved progress for each player, a Swing user interface, and an automated test suite.

The desktop game uses no network and stores nothing remotely, and there is no account system, multiplayer or sound.

A second phase has since been built: a Spring Boot and PostgreSQL leaderboard that scores submitted runs by replaying them, described in Section 7.7. A third produces an Android build from the same rules, described in Section 7.8. The Android app compiles and packages but has not yet been run on a device, which is recorded in Section 8.3.

All three guard types described in the original proposal are implemented: a patroller that walks a route, a hunter that pursues with A*, and a stationary sentinel whose vision cone sweeps a quarter-turn each turn. The sentinel is tested and can be placed from a level file, but no shipped level uses one: none of the ten has geometry in which a rotating cone leaves anywhere safe to wait, which is discussed in Section 8.3. Remaining gaps are recorded in Section 8.3 and in the accompanying Software Requirements Specification.

3. Software Requirements Specification

This section summarises the requirements. The full specification, with rationale for each requirement and a complete traceability matrix, is held in the repository as `docs/srs.md`.

Each requirement names the automated test that verifies it. Requirements marked **(D)** are derived: they were not in the original proposal and were added after testing or play revealed a fault.

3.1 Functional Requirements

Visibility and fog of war

ID	Requirement	Verified by
FR-1.1	Visible tiles are found by breadth-first flood fill from the player, limited to a sight radius of 5	`VisibilityServiceTest`
FR-1.2	Walls block the fill, so sight does not pass through them	`VisibilityServiceTest`
FR-1.3	A wall next to a visible tile is itself visible, so rooms appear enclosed	`VisibilityServiceTest`
FR-1.4	A tile once seen stays drawn as remembered terrain, without its occupants	`FogOfWarTest`
FR-1.5	A developer toggle reveals the whole map for testing	`FogOfWarTest`

Turns and movement

ID	Requirement	Verified by
FR-2.1	A turn is one player action followed by one move from every guard	`TurnWithGuardsTest`
FR-2.2	The player moves one adjacent tile per turn, or waits	`GameStateTest`
FR-2.3	A move into a wall or off the grid is refused and costs no turn	`GameStateTest`
FR-2.4	A move onto a guard is refused; guards block movement	`TurnWithGuardsTest`
FR-2.5	Each level has a turn limit, after which the attempt fails	`TurnLimitTest`

Guards, detection and capture

ID	Requirement	Verified by
FR-3.1	A patroller walks a fixed route, reversing at each end	`GuardTest`
FR-3.2	A hunter moves towards the player's last known position using A*	`GuardTest`, `AStarPathfinderTest`
FR-3.3	A guard is in exactly one of three modes: PATROL, ALERT, HUNT	`GuardTest`
FR-3.4	**(D)** A hunter moves two tiles on alternate turns, so it gains on a	`GuardTest`

	fleeing player	
FR-3.5	A sentinel never leaves its post and never hunts, so it cannot capture anybody	`SentinelTest`
FR-3.6	A sentinel sees only a ninety-degree cone in the direction it faces	`SentinelTest`
FR-3.7	A sentinel turns one quarter clockwise each turn, repeating every four turns	`SentinelTest`
FR-3.8	A sentinel keeps sweeping through an alarm, so its rotation can always be timed	`SentinelTest`
FR-4.1	A turn ending inside a guard's line of sight counts as a sighting	`SpottingTest`
FR-4.2	A sighting raises an alarm, which puts every hunter into HUNT mode	`AlarmTest`
FR-4.3	Sightings are counted for the whole run, not per attempt	`ScoreTest`
FR-5.1	A guard in HUNT mode reaching the player captures them	`CaptureTest`
FR-5.2	**(D)** A guard not in HUNT mode blocks the player but does not capture	`CaptureTest`
FR-5.3	A run starts with three lives; a capture costs one	`LivesTest`
FR-5.4	A new life resets player, keys, guards and clock, but not the sighting count	`LivesTest`

Keys, undo and hints

ID	Requirement	Verified by
FR-6.1	A level may hold several keys, and all are required	`MultipleKeysTest`
FR-6.2	The exit stays closed until every key is held	`MultipleKeysTest`
FR-7.1	Every turn is recorded as a command object on a stack	`UndoTest`
FR-7.2	Undo restores player, guards, alarm, keys and clock	`UndoTest`
FR-7.6	**(D)** Undo does not restore the sighting count; being seen is permanent for the run	`SpottingTest`, `AlarmTest`
FR-7.3	Undo repeats back to the start of the attempt, and is refused once it has ended	`UndoTest`
FR-8.1	A hint shows one move, never a route, and expires when the player moves	`HintTest`, `HintLifetimeTest`
FR-8.2	When the player is in a guard's sight, the hint points towards cover	`HintCoverTest`
FR-8.3	**(D)** A level allows three hints, and neither undo nor a new life restores one	`HintBudgetTest`

Scoring, levels and saved progress

ID	Requirement	Verified by
FR-9.1	Escaping scores one star, escaping within par a second, escaping unseen a third	`ScoreTest`
FR-9.2	Par is calculated from the level's own geometry, not written into the level file	`ScoreTest`
FR-9.3	**(D)** Every level allows a first turn that ends unseen, so the stealth star survives the opening	`StarRatingAchievableTest`
FR-9.4	**(D)** On level 1, exhaustive search confirms the fastest unseen escape fits within par, so all three stars are reachable together	`StarRatingAchievableTest`
FR-10.1	Ten levels are provided, and every key and the exit are reachable in each	`LevelIntegrityTest`
FR-10.2	**(D)** Every watched tile has cover within reach, so being seen can always be survived	`LevelFairnessTest`
FR-10.3	Levels unlock at 0, 1, 3, 5, 8, 11, 14, 17, 20 and 23 stars	`LevelIntegrityTest`
FR-11.1	Progress is saved under a player name, keeping the best result per level	`ProgressTest`
FR-11.2	Saving one player's progress does not affect another's	`ProgressTest`
FR-11.3	Progress is written only to the local machine	`ProgressTest`

3.2 Non-Functional Requirements

ID	Category	Requirement	Verified by
NFR-1	Performance	A turn resolves quickly enough to feel instant on a 29x21 grid	manual
NFR-2	Maintainability	No class in the six rule packages imports `java.awt`, `javax.swing` or `java.applet`	`ArchitectureTest`
NFR-3	Testability	The whole test suite runs without opening a window	`ArchitectureTest`
NFR-4	Reliability	The game is deterministic: the same input from the same state gives the same result	`TurnWithGuardsTest`
NFR-5	Portability	`ProgressStore` takes its storage path from the caller	`ArchitectureTest`

NFR-6	Privacy	The game sends no data anywhere and needs no network	by inspection

NFR-2 and NFR-4 are the two that carry the most weight. NFR-2 is what allows the test suite to run without a display, and what would allow the rules to be reused on another platform. NFR-4 is what makes the exhaustive search in Section 6.4 possible, and therefore what makes FR-9.3 and FR-10.1 checkable rather than merely claimed.

3.3 Hardware and Software Requirements

	Requirement
Java runtime	JDK 17 or later
Operating system	Any supporting Java SE with a display; developed on Windows 11
Runtime libraries	None beyond the Java standard library
Build tool	Apache Maven 3.6 or later
Test framework	JUnit 5, test scope only
Version control	Git and GitHub
Processor	Any processor supporting a Java 17 runtime
Memory	512 MB available to the JVM
Display	1024 x 700 or larger
Storage	About 5 MB for the application, under 10 KB for saved progress
Input	Keyboard; the mouse is used only on the menu screens

The display figure is a real lower bound. The board is deliberately not scaled down to fit a smaller window, because reducing the tile size makes the puzzle harder to read without making it easier to solve.

4. System Analysis

4.1 Existing System

There is no single system that Signal Lost replaces. What exists is a set of common approaches to building games of this kind, each with weaknesses that this project was designed to avoid.

Real-time architecture. The usual approach to a stealth game is a continuous simulation driven by a frame loop, with floating-point positions, interpolated movement and detection evaluated by casting rays many times a second. Three engineering consequences follow. The logic cannot be separated from the rendering, because state advances as a function of elapsed frame time. Behaviour is not reproducible, because the same inputs produce slightly different results on different hardware. And the design cannot be drawn honestly, because the rules are spread across update methods rather than held in identifiable objects.

Visibility by ray casting. Line of sight is usually found by casting rays from the observer and recording what each one hits first. This is fast, but it computes line of sight literally, so visibility leaks through the diagonal gaps between wall corners, and walls bounding a lit room are often left dark so that rooms do not appear enclosed. A large body of corrective algorithms exists for exactly these artefacts, which suggests the underlying model fits a grid imperfectly.

Verification by play. A level is usually validated by its author completing it. This shows that one person can win once. It does not show that the level can be won in general, that its difficulty rating can be reached, or that it is not in fact impossible. Unit tests, where they exist, usually check that individual rules behave correctly, which is a much weaker claim.

4.2 Proposed System

Problem	Solution in Signal Lost	Enforced by
Logic cannot be tested without the UI	All game logic is in seven packages that reference no user-interface type	`ArchitectureTest`
Behaviour is not reproducible	A discrete, deterministic turn model with no dependence on elapsed time	`TurnWithGuardsTest`
Patterns are obscured by the frame loop	Command, Strategy, State, Observer and Factory, each an identifiable set of classes	Section 5.3
Ray casting leaks through corners	Visibility as breadth-first traversal of the grid graph	`VisibilityServiceTest`
Playability is only claimed	Every key and the exit are proved reachable on all ten levels, and the game tree is searched exhaustively where it is small enough	`LevelIntegrityTest`, `StarRatingAchievableTest`
Difficulty thresholds may be unreachable	Star thresholds derived from level geometry and checked attainable	`StarRatingAchievableTest`

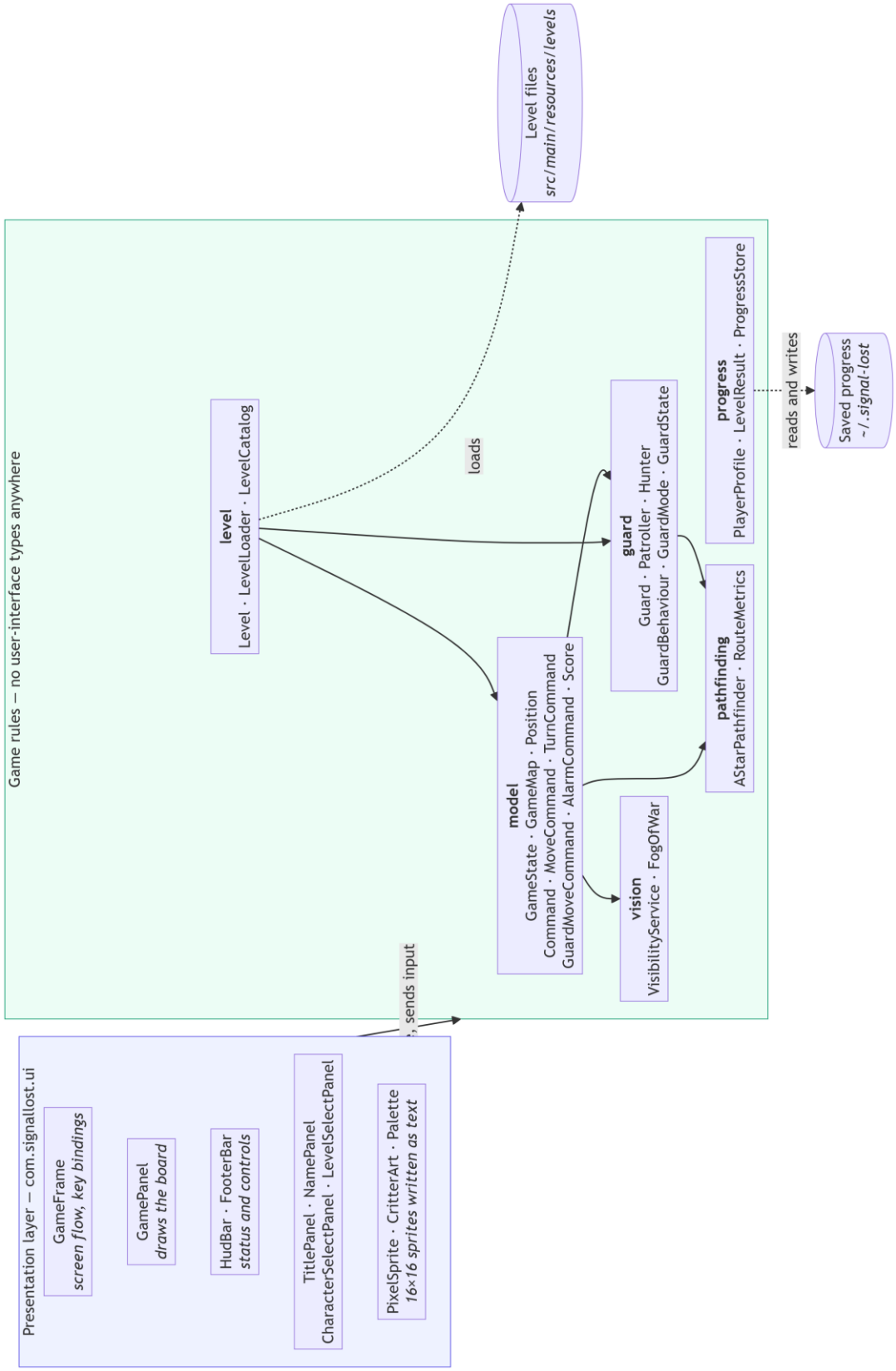
The main advantages are that the rules can be tested in isolation, that a failing scenario can be written down as a sequence of moves and reproduced exactly, that visibility behaves correctly at corners without special cases, that playability is demonstrated rather than assumed, and that moving the game to another platform means writing a new interface rather than a new game.

Two costs were accepted. A turn-based model gives up the tension of continuous movement. And visibility is recomputed each turn rather than updated incrementally, which is more work per turn than necessary. Neither matters at this grid size.

5. System Design

The diagrams are generated from Mermaid source held in `docs/uml.md`, so they are version-controlled as text and cannot drift away from the code.

5.1 System Architecture

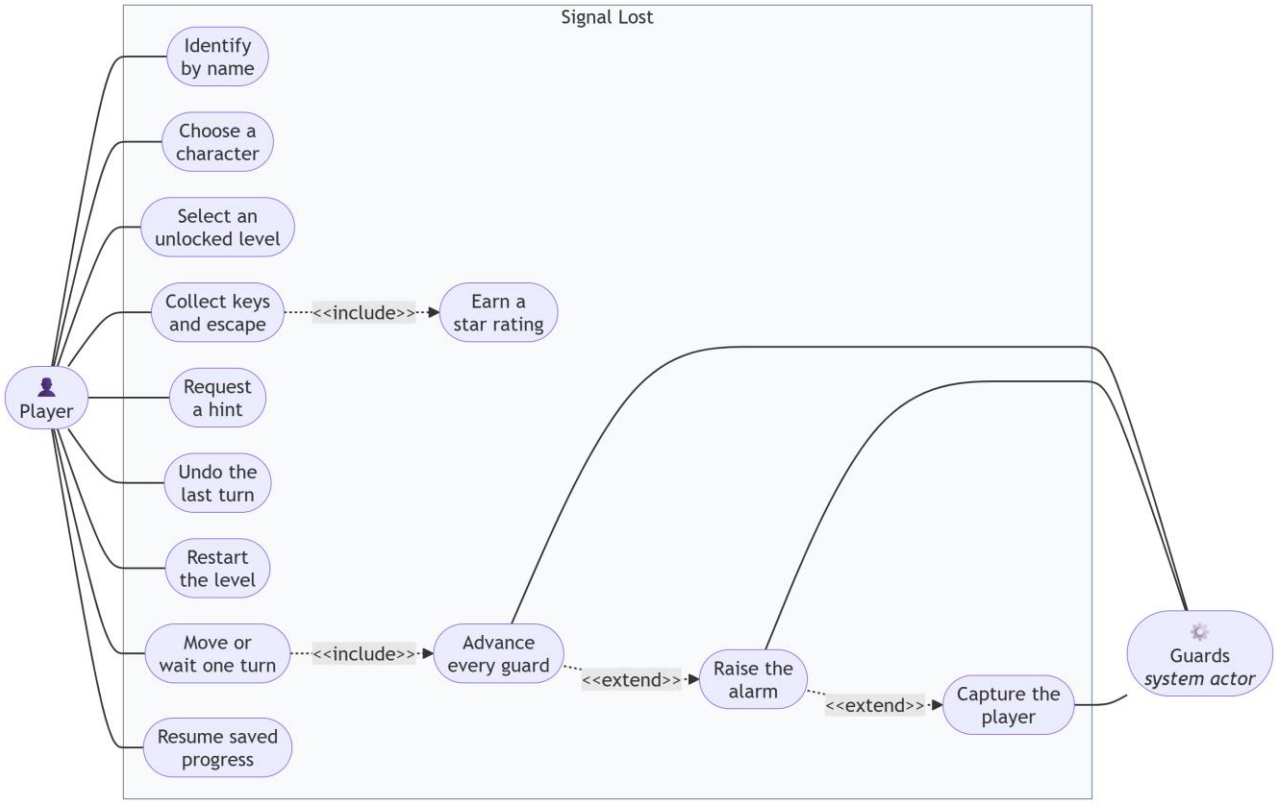


The system has two layers with one permitted direction of dependency. The presentation layer reads game state and forwards input. The rules layer knows nothing about it.

This is enforced rather than conventional. `ArchitectureTest` scans every source file in the six rule packages and fails the build if any of them imports `java.awt`, `javax.swing` or

`java.applet`. The test was added after seeing how easily the property could be lost: one model class using `java.awt.Color` as a convenient way to hold a colour would compile, pass every other test, and tie the rules permanently to the desktop.

5.2 Use Case Diagram

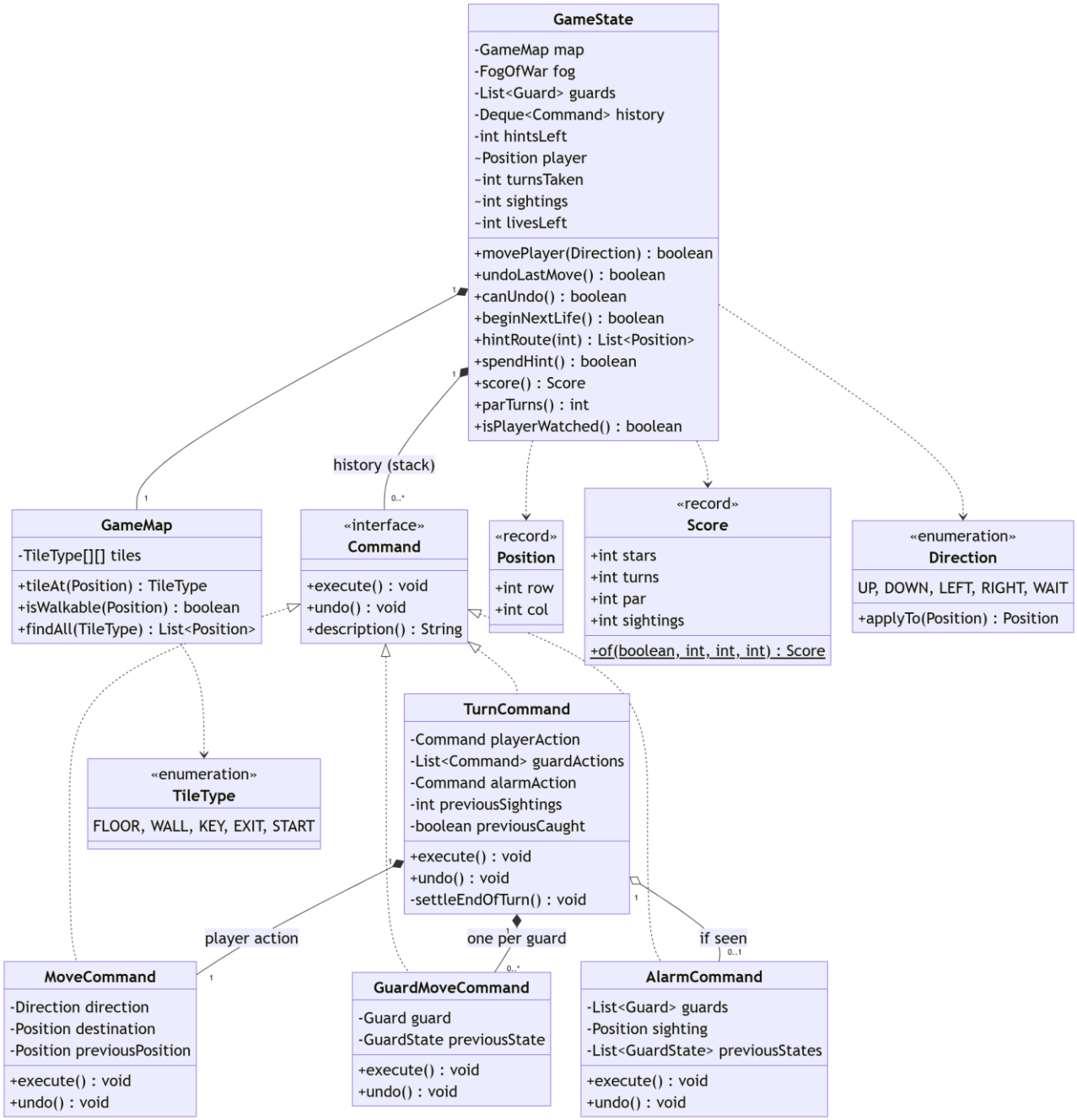


The player is the only human actor. Guards are a system actor, because they act every turn without being invoked and because two of the most important events in the game, the alarm and a capture, start with them. The include relationship from *Move or wait one turn* to *Advance every guard* captures the central rule: there is no player action that does not also give every guard a move.

5.3 Class Diagram

The class model is shown in three parts for legibility.

5.3.1 Core model and the Command pattern



`GameState` holds the mutable position of play. Its mutable fields are package-private, so only command objects in the same package can write to them.

The Command pattern is used at two levels. A `TurnCommand` represents one whole turn and owns the smaller commands that make it up: a `MoveCommand` for the player, a `GuardMoveCommand` for each guard, and an `AlarmCommand` if the player was seen. Undoing a turn means undoing each part in reverse order, so nothing can be missed.

5.3.2 Guards: the three types and their shared state

Every guard is the same thing from outside: something with a position, a mode and a set of tiles it can see. `GuardState` is an immutable record holding that whole condition. A guard does not

change itself; it calculates a new state and has one applied. This is what makes undo exact, because `GuardMoveCommand` restores the previous record rather than trying to reverse each field.

Sight is a single method on `Guard`, so the sentinel narrows it to a cone by overriding that method rather than by introducing a new concept. Everything that asks what a guard can see — the red overlay, the spotting check, the level fairness test — goes through it, and none of them needed changing when a third guard type was added.

5.3.2.1 Guards: Strategy and Observer

Three patterns meet at the guard. `GuardMode` is the State and decides which behaviour applies. `GuardBehaviour` is the Strategy, with four stateless implementations selected by that mode. `AlarmListener` is the Observer, through which one guard's sighting reaches every other guard without any of them holding a reference to the others.

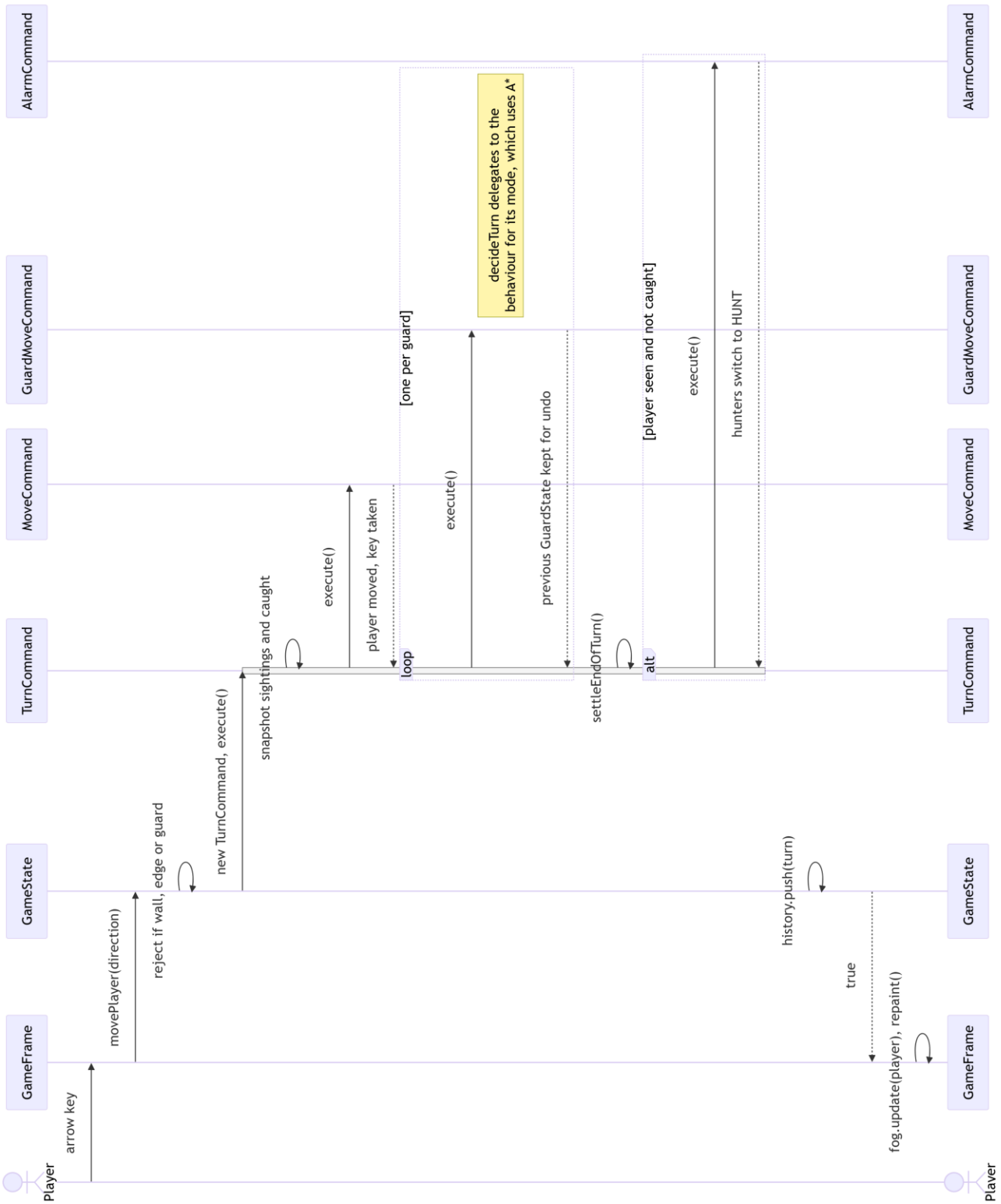
5.3.3 Vision, levels and progress

`VisibilityService` is stateless and calculates what is visible from a position. `FogOfWar` adds memory, classifying each tile as `VISIBLE`, `REMEMBERED` or `UNEXPLORED`. Keeping them separate makes the geometry testable on its own.

`LevelLoader` and `GuardFactory` form the Factory pattern, building a level and its guards from a text file so that adding a level needs no code change. `ProgressStore` is the only class in the rules layer that touches the file system, and it takes its path from the caller.

5.4 Sequence Diagram

5.4.1 Taking one turn



This is the system's main flow. The ordering is where the difficulty of the game comes from: the player acts, then every guard acts, and only then is it decided whether the player was seen. Moving

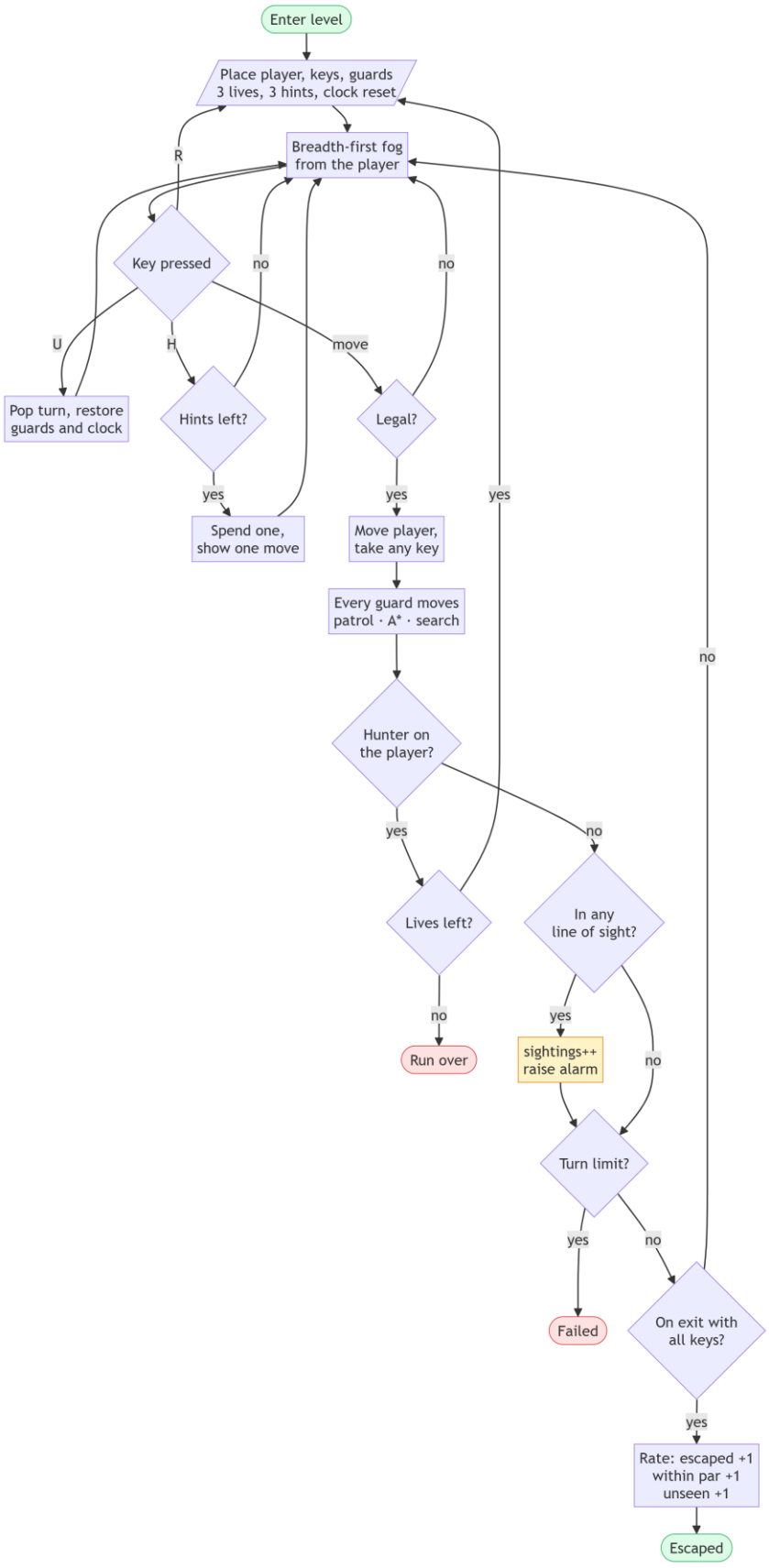
to a tile that is safe when you leave is therefore no guarantee, because a guard may turn a corner and be looking at it by the time the turn ends.

5.4.2 Undoing a turn

Undo pops the turn and reverses its parts in the opposite order: the alarm first, then each guard, then the player. Nothing is recalculated; everything is restored from values captured at the time.

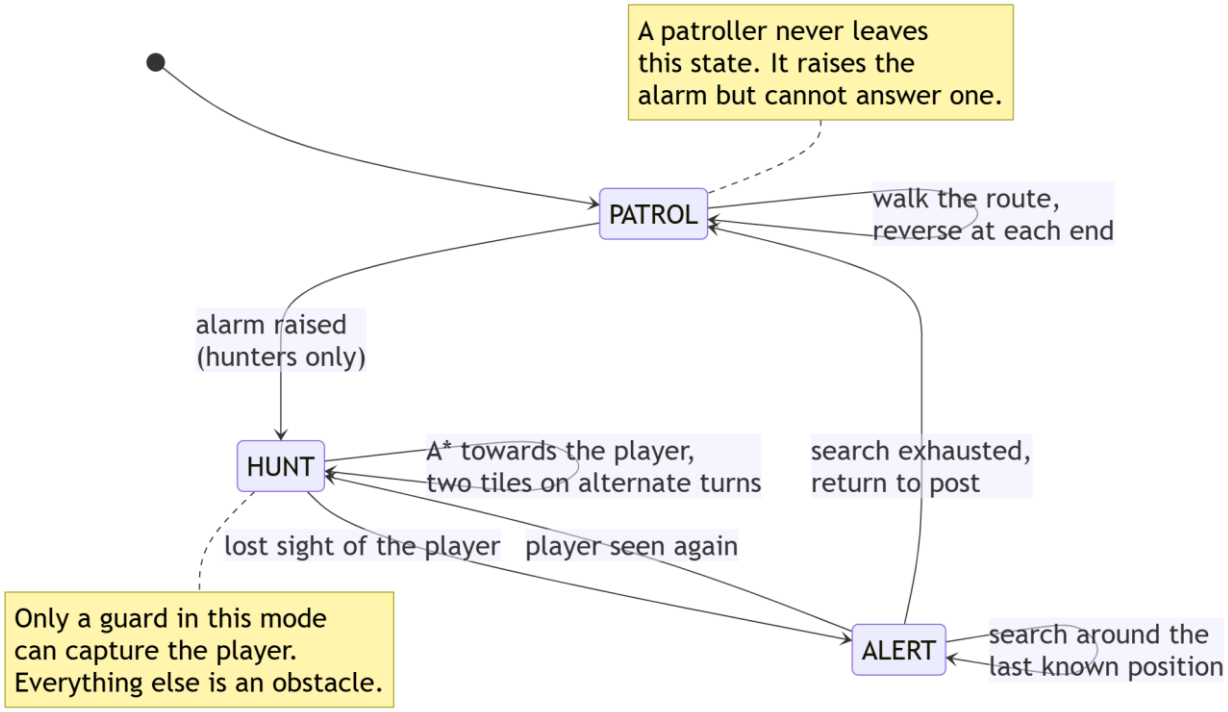
Two omissions are deliberate. The hint budget is not restored, because if it were, three hints would be unlimited: a player could ask, undo, and repeat. Nor is the sighting count, for the same reason — refunding it made the stealth star free to anyone willing to walk into a guard's sight and press U until a safe route turned up.

5.5 Activity Diagram



The diagram follows one attempt at a level from entry to rating, including undo, hints and restart, and both ways of failing: capture, which costs a life, and the turn limit, which ends the attempt.

5.6 Guard State Machine



Only a guard in HUNT mode can capture the player. A patroller that walks into the player blocks them and nothing more. This rule was adopted after the stricter version made a single-width corridor on level 1 impossible to pass, which made the level unwinnable.

5.7 Deployment View

The seven rule packages are compiled once and used by four programs that share no other code: the desktop game, a browser build compiled to JavaScript, an Android app, and a server that scores submitted runs. None of them holds a copy of the rules, and nothing in the rules knows that any of them exist.

Program	Front end	How it loads levels	Where progress goes
Desktop	Swing	classpath resources	file in the user's home
Browser / installable app	HTML canvas	fetched over the network	browser local storage
Android	`View` and `Canvas`	classpath resources	the app's private directory
Leaderboard service	none	classpath resources	PostgreSQL

This is the architecture of Section 5.1 taken to its conclusion. The dependency rule was stated there as a constraint and enforced by a test; here it is what makes three further programs possible without touching the first.

6. Methodology

6.1 Development approach

Development was incremental, with each stage producing a working program rather than a layer of an unfinished one.

Stage	Delivered
1	Grid model, breadth-first fog of war, playable board
2	Command pattern, undo stack, six characters
3	A* pathfinding, patrolling and hunting guards, hints
4	Detection, line-of-sight overlay, turn limit
6	Ten levels, title screen, player names, saved progress
7	This report, the SRS and the UML diagrams
5	The sentinel guard, built last

Stage 5 was deferred in favour of stage 6, on the judgement that a complete game with two guard types shows more than an incomplete one with three, and built once the rest was finished. Taking it last turned out to be the cheaper order: by then `GuardBehaviour` and `GuardFactory` had settled, so the sentinel was an addition rather than a change. Stage 8, the Spring Boot leaderboard, is phase 2.

6.2 Version control

Git was used throughout and the repository is published on GitHub. There are thirty-eight commits, each limited to one concern.

6.3 Testing strategy

Testing was done at four levels:

8. **Unit tests** on individual rules, such as visibility geometry, pathfinding and scoring.
9. **Integration tests** on whole turns, where the player acts, guards respond and the outcome is checked.
10. **Property tests** across all ten levels at once: that every key is reachable, that every watched tile has cover within reach, and that three stars can be achieved.
11. **Architectural tests** on the source itself, failing the build if the dependency rule in Section 5.1 is broken.

The order of events was often the reverse of the textbook one. Several of the most significant faults were found by playing rather than by testing: a hunter that escorted the player instead of catching them, a level whose only key stood behind a door that never opened, and a star rating that could not be achieved. In each case a test was written afterwards to prevent that kind of fault returning. The test suite is therefore a record of mistakes already made as much as a specification written in advance. The property tests at level 3 exist precisely because unit tests had proved unable to find faults of that kind.

6.4 Verification by exhaustive search

Because the game is deterministic, the question "can this level be won?" has a definite answer. A solver was written that performs a breadth-first search over the whole game tree, with each state identified by player position, turn count and the set of keys still to collect.

The solver established three things that testing had not. Level 1 was unwinnable with both guards in place, which no amount of play would have distinguished from a level that was merely hard. The star thresholds were unreachable: on level 1 the fastest escape took 38 turns and the fastest unseen escape took 56, against a par of 51, so the speed star and the stealth star excluded each other. And the solver itself was wrong twice before it was right. One version stored states without the turn count, which is unsound while a clock is running, and a later one recorded how many keys were held rather than which, so holding the first key and holding the third looked identical. Both versions reported winnable levels as lost.

That last point is worth noting. A verification tool is software too, and is no more trustworthy than the software it checks until it has itself been checked.

The search does not extend to the whole game. Pruning every branch in which the player is seen is what keeps it tractable, and that pruning is exactly what a proof of general winnability may not do, because being seen is a setback rather than a loss. Removing it was attempted and the search exhausted a three-gigabyte heap before reaching a verdict: the reachable state space of a fifteen-row level with three guards, each carrying its own position, facing and mode, is far larger than level 1's. What is proved for every level is therefore weaker and honestly stated — that each key and the exit are reachable on the static map (`LevelIntegrityTest`), that cover always lies within reach of a hunter (`LevelFairnessTest`), and that a first move exists which ends unseen (`StarRatingAchievableTest`).

A cheaper substitute was written in its place. `HintAutopilotTest` plays each level by following the hint every turn, which is not a proof of winnability but is a strong check on the advice: it asserts on all ten levels that every suggestion is a move the game will accept and that the advice never paces between two tiles. Both assertions failed when first written, and Section 7.9 describes the three faults they exposed.

7. Implementation

7.1 Module organisation

Package	Lines	Responsibility
`model`	1,171	Game state, map, commands, undo stack, scoring
`guard`	836	Guard types, modes, behaviours, the alarm
`progress`	284	Player profiles and saved progress
`level`	258	Level and catalogue loading from text
`pathfinding`	213	A* search and route metrics
`vision`	189	Visibility and fog memory
Rules total	**3,705**	**No user-interface dependency**

The four programs built on those rules, none of which the rules know about:

Module	Lines	Responsibility
`ui`	3,039	Swing front end for the desktop
`android`	1,958	Android front end (Section 7.7)
`web`	1,519	Browser and installable web app (Section 7.6)
`server`	762	Spring Boot leaderboard (Section 7.8)

7.2 Fog of war

Visibility is a breadth-first flood fill from the player's tile, bounded by the sight radius and blocked by walls. Walls next to a visible tile are marked visible in the same pass, so rooms appear enclosed.

A flood fill was chosen over ray casting. Because it spreads only between adjacent tiles, visibility cannot leak through the diagonal gap between two wall corners, which ray-cast implementations have to correct for explicitly. Sight bending around a corner follows naturally from the traversal.

7.3 Pathfinding and pursuit

A* is used over the grid graph, with a Manhattan distance heuristic and a priority queue ordered by estimated total cost. Ties are broken deterministically so identical situations always resolve identically.

A second version of the method accepts a cost for each tile. This is used by the hint system rather than by the guards: tiles under observation cost more to cross, and tiles near a hunting guard more still, so the route returned avoids being seen rather than being shortest.

A hunter moving at exactly the player's speed can never close a gap. The first version did this, so a chase stayed at whatever distance it started and the hunter appeared to accompany the player rather than chase them. The fix was to let a hunter take two steps on alternate turns, giving a net gain of half a tile per turn. Running in a straight line now loses, and breaking line of sight wins. A hunter

that always took two steps was tried first and proved impossible to escape, because capture arrived in four turns where the nearest cover was five away.

7.4 Undo

Every turn is a `TurnCommand` holding a `MoveCommand`, one `GuardMoveCommand` per guard, and an `AlarmCommand` where one was raised. Each sub-command keeps the state it replaced. Guards keep an immutable `GuardState` record holding position, mode, route index, last known player position, remaining alert turns and surge status.

Undo therefore restores the guards, the alarm, the keys and the clock, not only the player's position. What makes this exact is that guards are never changed field by field, only replaced whole.

7.5 Saved progress and artwork

Progress is stored as a properties file in the user's home directory, keyed by player name, holding the best result for each level. Writing merges rather than overwrites, so one player's progress cannot disturb another's. A file that is missing, empty or damaged is treated as "no progress" rather than shown as an error. `ProgressStore` takes its path from the caller, so a platform without a home directory can supply its own.

All sprites are 16x16 pixel art written as arrays of strings in Java source, where each character names a colour supplied by the character that owns the sprite. The same shape data therefore produces a ginger cat and a pink rabbit. Artwork lives in the repository as readable text and can be reviewed like any other source.

7.6 The browser build and the installable web app

The rules were compiled to JavaScript with TeaVM and given a canvas front end, so the same Java that runs on the desktop runs in a browser. Two things differ and both are handled at the edge rather than by changing a rule. A browser has no classpath, so levels arrive over the network and are parsed by `LevelLoader.parseLevel` instead of its resource loader; and it has no filesystem, so progress is kept in local storage rather than a properties file. The same decision Android made when it handed `ProgressStore` a different directory, taken one step further.

A web application manifest and a service worker turn the page into something a player can add to a home screen and open offline. This matters for reach rather than polish: Apple does not permit an Android package to be installed, so a browser is the only way the game reaches an iPhone at all.

The service worker is also where the sharpest deployment lesson of the project came from. Its first version cached under a fixed name, which meant the worker file was byte-identical on every deployment. A browser decides whether to install a replacement worker by comparing that file against the one it holds, so no replacement was ever installed, and every player who had opened the site once was served that first build permanently. The fix is that the build stamps a hash of everything it is about to cache into the worker, so a changed game guarantees a changed worker, and the page and game code are now fetched network-first with the cache as the offline fallback. A caching layer that cannot be updated is not a cache; it is a copy nobody can replace.

7.7 The Android build

A second interface onto the same game, 377 lines in two classes.

The Gradle module does not copy anything. Its source set points at the existing `src/main/java`, excludes the `ui` package, and compiles:

```
groovy srcDirs = ['src/main/java', '../../src/main/java'] exclude 'com/signallost/ui/**'
```

The resulting 56 KB APK contains the rules classes across all seven packages, all ten level files, three Android classes, and **no user-interface class from the desktop build**. Not one line of the rules was changed to produce it.

`GameActivity` and `BoardView` are the whole front end. There is no layout XML, no AndroidX and no third-party dependency: the board is drawn onto a `Canvas`, much as the desktop version draws onto a `Graphics2D`. Tapping anywhere on the board steps one tile in that direction, which keeps the turn-based feel without asking a thumb to hit a 40-pixel square; three strips along the bottom give undo, hint and restart, and the hardware back button undoes.

The only platform-specific concession in the rules layer is the one anticipated in the design: `ProgressStore` takes its path from the caller, so Android supplies `getFilesDir()` where the desktop supplies the home directory.

7.8 The leaderboard service

A Spring Boot and PostgreSQL service that records completed runs and serves a leaderboard, 762 lines across 15 files, verified by 18 tests. It is built but not deployed; Section 9.4 explains why.

The problem it solves. A leaderboard that accepts a score has to believe whatever the client sends, and the client runs on the player's machine. A request saying `{"stars": 3}` is a claim, not a fact. Obfuscating the client or checking a score for plausibility makes forgery inconvenient rather than impossible, because the number still originates somewhere the player controls.

What it does instead. The service accepts no score. A submission carries the player's name, the level, and the sequence of moves made, one letter per turn:

```
json {"playerName": "Uzma", "levelName": "level1", "moves": "DUDUDUDUDDDDDDDDRRRRUUUURRRRUUUURRRRDDDDDDDD"}
```

The server loads the level, replays those moves through the same `GameState` the game uses, and scores the result itself. A forged run must be a sequence of moves that genuinely survives the guards, which is indistinguishable from playing the level. This works only because the rules are deterministic and free of the user interface, so the server can run them with no screen attached — the architectural constraint of Section 5.1 turning out to buy something that was never the reason for it.

7.9 What playtesting found

The game was given to players outside the development, and what they reported could not have been found by the test suite as it stood.

The hint chose which key to walk towards by straight-line distance, which knows nothing about walls. Measured across every standable tile of every level it was wrong on nine of the ten: on level 5 the first hint a player spends points 23 steps away from the key they could actually reach, and on level 10 the gap reaches 56. Players reported this as "the hints are not proper" and as being unable to finish level 5, and both were the same defect. It now measures along the route it would actually draw. It also treated a guard as expensive ground rather than a closed door, while `movePlayer` refuses to walk into one, so when a guard stood in the only corridor the suggested move was one the game would not accept.

A life was deducted when the player acknowledged a capture rather than when the capture happened, so the banner announcing it still counted the life just lost, and a player caught for the first time was told they had three left. The existing test asserted this under the name "unspent until claimed", a reminder that a test records a decision rather than validating it.

What these have in common is that they live in the seam between the rules and the interface, which is the one place the architecture deliberately keeps the test suite out of. The rules were correct in every case. What was wrong was the advice given about them, and the moment a consequence was applied.

8. Output Screenshots

The images below are renders of the desktop build, produced by a tool that drives the real game with A* rather than scripted keystrokes, so they cannot fall out of step with the code.

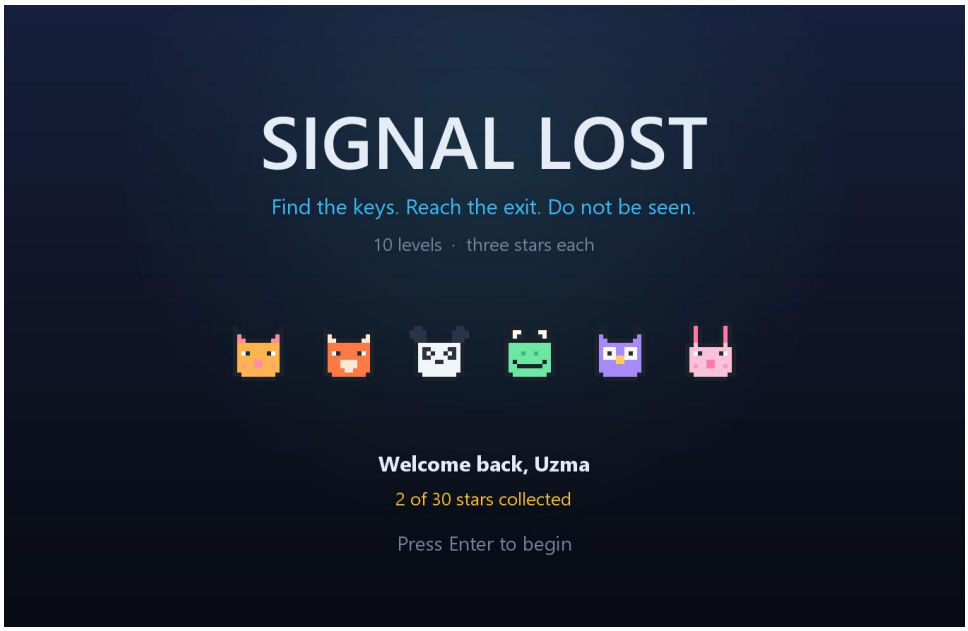


Figure 8.1 — Title screen. The game opens here, and greets a returning player by name with the stars they have collected so far.

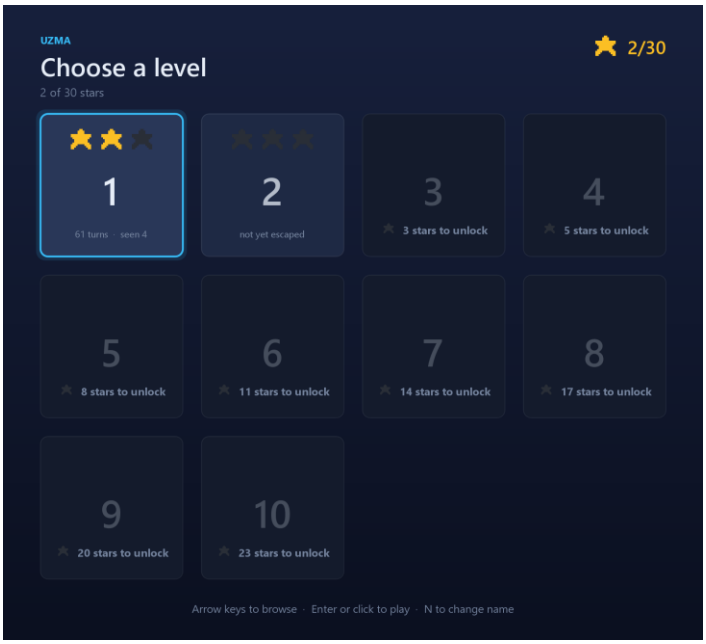


Figure 8.2 — Level selection. Levels unlock against accumulated stars, and each box shows the best rating achieved on it.

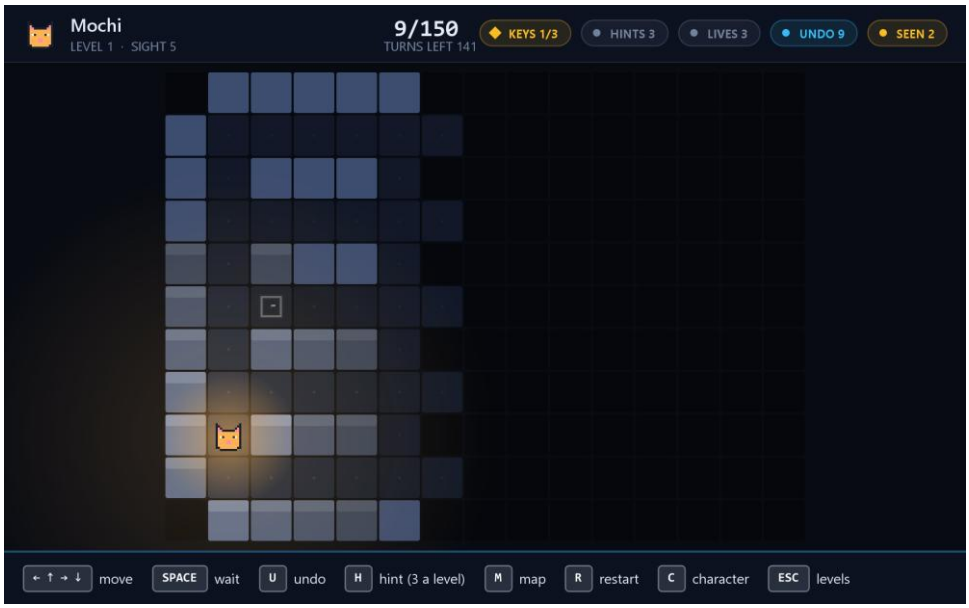


Figure 8.3 — Fog of war. Bright tiles are currently visible, dim tiles are remembered terrain, and black is unexplored.

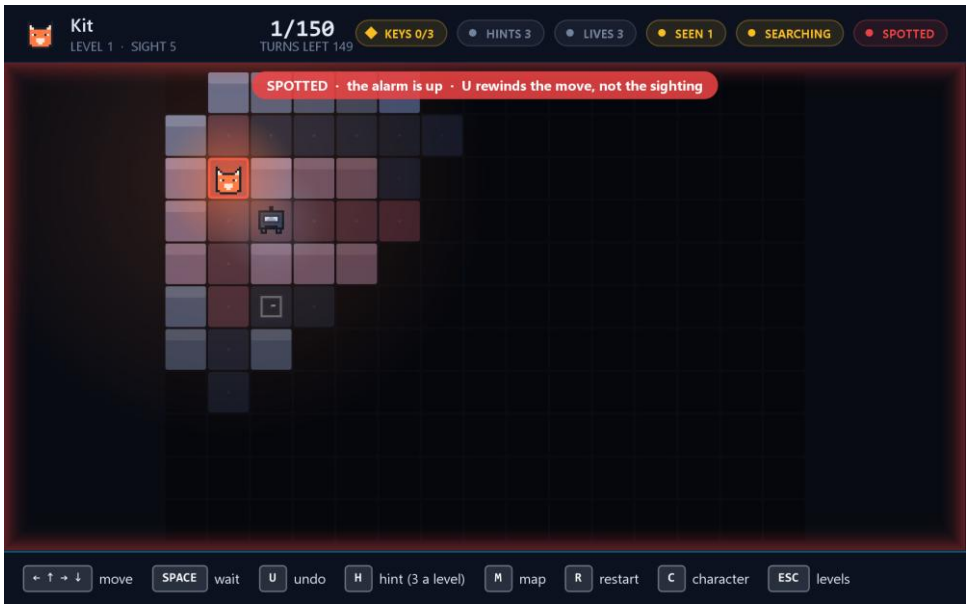


Figure 8.4 — Spotted. The guard's line of sight is drawn in red, the alarm is raised, and a hunter begins moving towards the sighting.

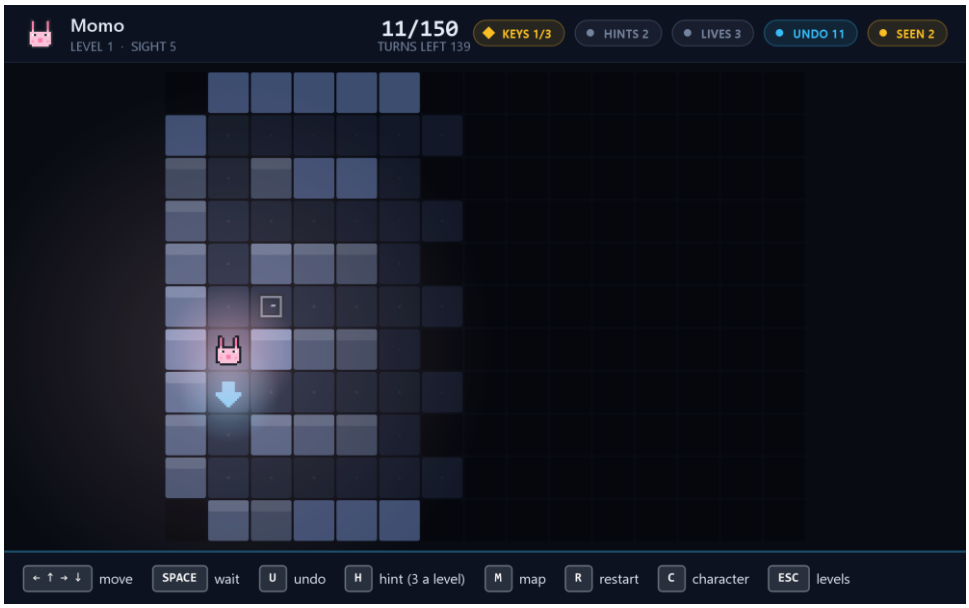


Figure 8.5 — A hint. One suggested move, shown for a single turn, routed to avoid being seen rather than to be shortest.

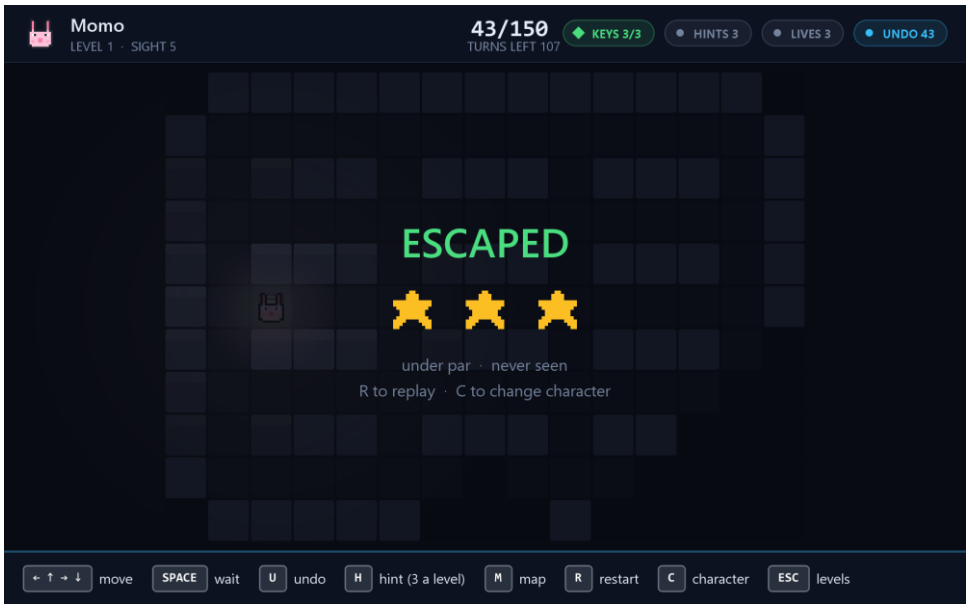


Figure 8.6 — Escaped. The rating screen, showing turns taken against par and the stars awarded.

9. Conclusion

9.1 Summary

Signal Lost was delivered as a complete, playable turn-based stealth game in Java, with ten levels, six characters, three guard types, exact undo, a limited hint system, star-based scoring and saved progress. The rules are 3,705 lines across seven packages and are verified by 260 automated tests in 31 test classes, with 18 more covering the leaderboard service: 248 in all.

A browser build, an Android build and a leaderboard service were added afterwards, all three reusing the rules without modification, so the same compiled logic now runs in four programs: a desktop game, a web page that installs to a home screen, a phone app, and a server.

All seven objectives in Section 2.2 were met. Fog of war is a bounded breadth-first flood fill, pursuit uses A* with a Manhattan heuristic and a priority queue, five design patterns are applied where they carry structural weight, the separation of rules from interface is enforced by a test, behaviour is verified by testing, playability is verified by search, and the result is a finished game.

9.2 What the project demonstrates

An architectural rule that nothing enforces will not survive. The separation between rules and interface is the project's most useful property, and also one that a single convenient import would quietly destroy. A test that fails the build on such an import turns an intention into a guarantee.

Passing tests and a working game are different claims. The most instructive fault was a level that could not be won while every test passed, because the tests checked that each rule behaved correctly and nothing checked that the game built from those rules could be finished. Tools need checking too: the solver written to validate the levels was itself wrong twice, each time reporting winnable levels as unwinnable.

9.3 Limitations

- The sentinel guard is implemented and tested but appears in no level. Placed in level 5, its six-tile cone covered fifteen of the sixteen walkable tiles in that room across one sweep, leaving nowhere to wait while it turned. The guard type is sound; the levels were not built for it.
- The Android build has not been run. It compiles, packages and contains the expected classes and levels, but no device or emulator was available to launch it on, so nothing is claimed about how it behaves.
- No performance budget is measured; NFR-1 is claimed rather than verified.
- Nothing in the game teaches its rules. A new player learns that red tiles are watched, and that only hunters capture, by being caught.
- No accessibility conformance is claimed. The line-of-sight overlay in particular conveys meaning by colour alone.
- Automated testing of the interface covers navigation and key bindings only; nothing checks what is drawn.

9.4 Future work

Term	Work
Near	An in-game tutorial; a colour-blind-safe line-of-sight overlay
Phase 2	Built, Section 7.7. Still to do: deploy it, and add the daily level
Phase 3	Built, Section 7.8. Still to do: run it on a device, port the pixel sprites, and add the level select
Longer	Have both clients submit their runs to the leaderboard, which needs only the move sequence they already have

10. References

12. Hart, P. E., Nilsson, N. J. and Raphael, B. (1968). "A Formal Basis for the Heuristic Determination of Minimum Cost Paths." *IEEE Transactions on Systems Science and Cybernetics*, 4(2), pp. 100–107.
13. Dijkstra, E. W. (1959). "A Note on Two Problems in Connexion with Graphs." *Numerische Mathematik*, 1(1), pp. 269–271.
14. Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
15. IEEE Std 830-1998. *IEEE Recommended Practice for Software Requirements Specifications*. Institute of Electrical and Electronics Engineers.
16. Oracle Corporation. *Java SE 17 Documentation*. https://docs.oracle.com/en/java/javase/17/
17. The JUnit Team. *JUnit 5 User Guide*. https://junit.org/junit5/docs/current/user-guide/
18. Mermaid. *Diagramming and charting tool*. https://mermaid.js.org/
19. Yasmeen, U. (2026). *Signal Lost — source repository*. https://github.com/Uzma-Yasmeen/SignalLost